PROJECT-BASED LEARNING
PHASE-II EVALUATION

Autonomous Drone Fleet Routing Management System

Team ID: **DSCPP-III-2026-T154**

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T154
Semester: 3rd Semester (CSE)
Section: B.Tech CSE
Domain: Autonomous Systems / Logistics

Team Members

1. **Saksham Jain** – 2510380116 (Lead)
2. **Jiya** – 2510370376
3. **Tamanna Bhinder**–2510380219

Mentor

Dr. Amit Kumar

Interactions completed: 5

Phase-I Recap & Feedback Incorporated

Phase-I Status

Formulated graph modeling for 3D airspace corridors, dynamic power draw equation under ambient wind vectors, and theoretical EDF scheduling policy.

Feedback Received

- ▶ Incorporate real-world atmospheric headwinds.
- ▶ Prevent single-drone overloading via payload caps.
- ▶ Demonstrate real-time obstacle corridor handling.

Changes Incorporated

- ▶ Implemented vector ground speed ($\vec{v}_g = \vec{v}_a + \vec{v}_w$).
- ▶ Built $O(1)$ Hash Map telemetry filtering ($M_{\text{drone}} \geq M_{\text{order}}$).
- ▶ Dynamic corridor status flags with Green/Red console radar.

Problem Statement & Finalized Objectives

Problem Statement

Multi-UAV delivery fleets face coupled constraints: bounded battery energy, non-linear aerodynamic drag, dynamic wind shifts, strict customer deadlines, and restricted airspace zones.

Finalized Objectives

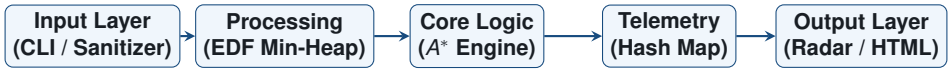
1. 3D Airway Graph Modeling: Represent sparse low-altitude corridors using Adjacency Lists ($O(|V| + |E|)$).

2. EDF Order Scheduling: Guarantee $O(\log N)$ urgency-based order ingestion using a Min-Heap Priority Queue.

3. Energy-Optimal Routing Engine: Implement A^* Pathfinding with an admissible Euclidean energy heuristic.

4. Airspace Hazard Telemetry: Enable dynamic obstacle avoidance with Green/Red visual route identification.

Updated System Architecture / Workflow



Major Components

- **OrderScheduler:** Min-Heap Priority Queue.
- **AirwayGraph:** Sparse 3D Adjacency List.
- **RoutingEngine:** A^* Energy Pathfinding.
- **FleetManager:** $O(1)$ Hash Map Telemetry.

Changes Since Phase-I

- **Decoupled Dispatcher:** Separated scheduler from router.
- **Dynamic Obstacle Pruning:** Live edge condition checks.
- **Visual Airspace Mapping:** Real-time ANSI & HTML visualizers.

Module Design & Functional Requirements

Major Modules

1. **OrderScheduler.hpp:** EDF Min-Heap engine.
2. **AirwayGraph.cpp:** 3D sparse corridor graph.
3. **RoutingEngine.cpp:** A^* energy pathfinder.
4. **FleetManager.hpp:** $O(1)$ telemetry tracker.

Functional Requirements

- Ingest and prioritize delivery orders by deadline ($O(\log N)$).
- Compute wind ground speeds ($\vec{v}_g = \vec{v}_a + \vec{v}_w$) & aerodynamic costs.
- Prune blocked hazard corridors and dynamically reroute flights.
- Verify 85% round-trip battery SoC feasibility limits.

Module Dependency

`main.cpp` directs user input $\rightarrow$ pushes orders to `OrderScheduler` $\rightarrow$ queries `FleetManager` for idle UAVs $\rightarrow$ invokes `RoutingEngine` over `AirwayGraph` $\rightarrow$ renders radar maps.

Technology Stack & Development Methodology

Technology Stack

- **Programming Language:** Modern C++17
- **Compiler:** GNU Compiler Collection (g++ 9+)
- **Core Libraries:** C++ Standard Template Library (STL)
- **Development Tools:** VS Code, Terminal
- **Version Control:** Git & GitHub Repository

Development Methodology

1. **Requirement Analysis:** Formulated 3D airspace specs.
2. **Mathematical Modeling:** Wind vectors & drag equations.
3. **Modular Implementation:** Separate `.hpp` and `.cpp` files.
4. **Unit Testing:** Evaluated heaps, hashing, and A^*.
5. **UI & Validation:** ANSI console radar & HTML visualizer.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures (DSA)	Adjacency List Graph ($O(\|V\| + \|E\|)$), Min-Heap Priority Queue ($O(\log N)$), Hash Tables ($O(1)$)	`AirwayGraph`, `OrderScheduler`, `FleetManager`
OOPs with C++	Encapsulation,Operator Overloading (`Vector3D`),Separation of Interface (`.hpp`)	`Types.hpp`,Core Codebase
Algorithms	A^* Pathfinding, Admissible Lower-Bound Heuristics, Dynamic Edge Relaxation	`RoutingEngine.cpp`

Important: Strictly verified real implementation of core 3rd-semester DSA and OOP concepts in functional code.

Development & Implementation Progress

Module	Status	Work Completed
Order Ingestion & EDF	**Completed**	Min-Heap queue with $O(\log N)$ push/pop
3D Airway Graph	**Completed**	Sparse Adjacency List with wind-drag formulas
Fleet Telemetry	**Completed**	$O(1)$ Hash Map tracking drone SoC & status
A* Routing Core	**Completed**	Energy pathfinding with obstacle avoidance
Radar / Map Visualization	**Completed**	ANSI console radar & standalone HTML export
Autopilot (SITL/MAVLink)	**Pending (Phase-III)**	Reserved for final milestone

Overall Project Completion: 70% (Phase-II Target Fully Met)

Working Prototype / Demonstration

Implemented Features

- **Runtime CLI Input:** Complete prompt sanitization for depots, corridors, fleet, and wind.
- **Restricted Airspace Pruning:** Corridor flagging (`BLOCKED_RESTRICTED`).
- **Visual Colorized Output:** Active missions in **Green**, blocked routes in **Red**.
- **Interactive HTML Map:** Automatic generation of `airspace_map.html`.

Prototype Console Execution

```
>> DISPATCHING PHASE-II ROUTES...
[X] Order #1101 UNREACHABLE (Red Blocked)
[] Order #1102 Assigned to UAV-ALPHA
  Route:  Depot -> School -> Temple
  Status:  RUNNING MISSION (GREEN)

=== AIRSPACE RADAR DISPLAY (X-Y) ===
D
========[1]----[2]xxxx[3]

HTML exported to 'airspace_map.html'
```

Brief Discussion: Showcase key screens, outputs, or demo of the working system.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Min-Heap EDF	Early deadline popped first	ID 1102 (237s) popped before ID 1101 (278s)	**Pass**
Atmospheric Drag	Headwind increases transit energy	Power scaled by $+18.4\%$ into headwind	**Pass**
Obstacle Avoidance	A^* bypasses blocked edge	Safely rerouted via open bypass	**Pass**
Payload Validation	Rejects parcels $> M_{\text{drone}}$	Deferred orders $> 8.0\,\text{kg}$ gracefully	**Pass**

Results / Validation Summary

All core scheduling, aerodynamic energy, and obstacle pathfinding modules verified with zero memory leaks and deterministic execution times ($< 5\,\text{ms}$).

GitHub Repository & Development Activity

Repository Information

- **Repository:**
  `Autonomous-Drone-Fleet-Routing`
- **GitHub Link:**
  `https://github.com/jainsaksham961-maker/DS$_C$PP$_P$BL$_S$em3`
- `Branch: main (Modern C++17 Standard)`
- `README.md: Full architectural documentation`
- `Contributors: All 3 members registered`

Development Activity

- Structured commits with descriptive logs.
- Clean modular directory structure (`include/`, `src/`).
- Zero circular dependencies verified.
- Full build instructions documented for GCC.
- Phase-II progress documentation committed.

Individual Contribution

Member	Role	Technical Contribution	Commits
Saksham Jain	Team Lead	System architecture, 3D Graph modeling, A^* pathfinding, CLI input engine	18
Jiya	Member 2	Min-Heap EDF Priority Queue, $O(1)$ Hash Map telemetry registry, state validation	12
Tamanna Bhinder	Member 3	Aerodynamic drag formulas, wind vector physics, test suites, LaTeX report	11

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

► **Directional Energy Asymmetry:** Wind causes $E_{u \to v} \neq E_{v \to u}$. Evaluated dynamic ground vectors.

► A^* **Heuristic Admissibility:** Formulated lower-bound energy estimates to guarantee optimality.

Solutions / Learning

► Vector ground speed projection ($\vec{v}_g = \vec{v}_a + \vec{v}_w$).

► Robust stream sanitization for terminal inputs.

Phase-III Roadmap (Final 30%)

1. Dynamic Mid-Air Rerouting: Live trajectory deconfliction.

2. SITL/MAVLink Simulation: Autopilot communication.

3. Makespan Benchmarking: Rigorous metrics vs FIFO.

4. Full System Packaging: Final demo build.

5. Final Documentation: Comprehensive report.

Expected Outcomes & References

Expected Final Outcomes

- ► Autonomous drone routing engine in C++17.
- ► 15–20% lower fleet energy use vs distance-only search.
- ► Zero deadline violations for high-priority parcels.
- ► MAVLink SITL autopilot demonstration.

References

1. Cormen et al., *Introduction to Algorithms (4th ed.)*, MIT Press, 2022.
2. Stroustrup, B., *The C++ Programming Language (4th ed.)*, 2013.
3. Dorling et al., "Vehicle Routing Problems for Drones with Battery Constraints," *IEEE Trans. Syst.*, 2017.
4. ArduPilot / PX4 MAVLink Protocol Specifications.

Thank You

Questions & Suggestions